

Lesson 6 Structured Note: Linked-List Stack Operations and Helpers

0. Current Position in the Course

You are now in **Lesson 6** of the stack curriculum.

This lesson belongs to:

Phase 2: Linked-List Stack

Before this lesson, you already learned:

Lesson	Main idea
Lesson 1	A stack follows LIFO: Last In, First Out .
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack functions.
Lesson 5	A linked-list stack uses nodes and a <code>top</code> pointer.

Now Lesson 6 continues the linked-list stack.

In Lesson 5, you learned the basic idea:

```
top -> 30 -> 20 -> 10 -> NULL
```

In Lesson 6, you learn the full set of linked-list stack operations:

- `push(x)`
- `pop()`
- `peek(pos)`
- `stackTop()`
- `isEmpty()`
- `isFull()`

The stack rule is still the same:

Last In, First Out

The implementation is different.

In an array stack:

top is an index

In a linked-list stack:

top is a pointer

That is the main shift.

1. Lesson 6 Goal

By the end of this lesson, you should understand:

- How linked-list `push()` works.
 - How linked-list `pop()` works.
 - How linked-list `peek(pos)` works.
 - Why linked-list `peek(pos)` needs traversal.
 - Why `peek(pos)` moves `pos - 1` times.
 - How `stackTop()` reads the top value.
 - How `isEmpty()` checks whether the stack has no nodes.
 - How `isFull()` checks whether heap memory is available.
 - Why `malloc()` is used during `push()`.
 - Why `free()` is used during `pop()`.
 - How to avoid memory leaks in `isFull()`.
 - Why `push`, `pop`, `stackTop`, `isEmpty`, and `isFull` are usually $O(1)$.
 - Why linked-list `peek(pos)` is $O(n)$.
-

2. Quick Review: What Is a Linked-List Stack?

A linked-list stack is a stack implemented using linked-list nodes.

A stack follows:

LIFO = Last In, First Out

This means:

The last value inserted is the first value removed.

Example:

```
push(10);  
push(20);  
push(30);
```

Logical stack:

```
Top -> 30  
      20  
      10
```

If we call:

```
pop();
```

The removed value is:

30

Because 30 was inserted last.

3. Node Structure

A linked-list stack is made of nodes.

Each node stores two things:

1. The data value.
2. The address of the next node.

In C:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Meaning of Each Field

Field	Simple meaning
data	Stores the actual stack value.
next	Stores the address of the next node.

Example node:

```
[ data = 10 | next = NULL ]
```

This node stores 10.

Its next is NULL, so there is no node after it.

4. The top Pointer

In an array stack, top was an integer index.

Example:

```
int top;
```

In a linked-list stack, top is a pointer to a node.

Example:

```
struct Node *top = NULL;
```

This means:

```
top currently points to nothing.
```

So the stack is empty.

Empty stack:

```
top -> NULL
```

Non-empty stack:

```
top -> 30 -> 20 -> 10 -> NULL
```

Here:

```
top points to 30
```

So `30` is the top element.

5. Array Stack vs Linked-List Stack

Concept	Array stack	Linked-list stack
Storage	Array	Nodes
Meaning of <code>top</code>	Index	Pointer
Empty condition	<code>top == -1</code>	<code>top == NULL</code>
Full condition	<code>top == size - 1</code>	<code>malloc()</code> returns <code>NULL</code>
Push location	Right side/end of array	Front of linked list
Pop location	Right side/end of array	Front of linked list
Fixed capacity?	Yes	No fixed capacity at creation

Important idea:

```
The stack rule does not change.  
Only the storage method changes.
```

Both array stacks and linked-list stacks follow LIFO.

6. Empty Condition in a Linked-List Stack

A linked-list stack is empty when:

```
top == NULL
```

Why?

Because `top` is a pointer.

If `top` points to nothing, there is no first node.

Example:

```
top -> NULL
```

This means:

```
The stack has no elements.
```

7. `isEmpty()` Operation

Meaning

`isEmpty()` checks whether the stack has no nodes.

It answers this question:

```
Is the stack empty?
```

Code

```
int isEmpty() {  
    return top == NULL;  
}
```

Return Values

Return value	Meaning
1	Stack is empty.
0	Stack is not empty.

Example 1

Given:

```
top -> NULL
```

Then:

```
isEmpty();
```

returns:

```
1
```

Because the stack is empty.

Example 2

Given:

```
top -> 30 -> 20 -> NULL
```

Then:

```
isEmpty();
```

returns:

```
0
```

Because the stack has nodes.

Complexity

`isEmpty()` is:

`O(1)`

Reason:

It only checks one pointer:

`top == NULL`

It does not scan the list.

8. Full Condition in a Linked-List Stack

In an array stack, the stack is full when:

`top == size - 1`

But a linked-list stack does not have a fixed array size.

A linked-list stack grows by creating new nodes in heap memory.

So the stack is full only when the heap cannot provide memory for a new node.

This is checked using:

`malloc()`

If `malloc()` fails, it returns:

`NULL`

So in a linked-list stack:

Stack full means memory allocation failed.

9. `isFull()` Operation

Meaning

`isFull()` checks whether a new node can be created.

It answers this question:

Is heap memory available for one more node?

Code

```
int isFull() {
    struct Node *t = (struct Node *)malloc(sizeof(struct Node));

    if (t == NULL) {
        return 1;
    }

    free(t);
    return 0;
}
```

Code Explanation

Step 1: Try to create one test node

```
struct Node *t = (struct Node *)malloc(sizeof(struct Node));
```

This asks the heap for memory for one node.

Step 2: Check if allocation failed

```
if (t == NULL) {  
    return 1;  
}
```

If `t == NULL`, the heap could not give memory.

So the stack is full.

Step 3: Free the test node if allocation succeeded

```
free(t);
```

This is very important.

If allocation succeeds, we only used that node for testing.

We do not actually want to insert it into the stack.

So we must release it.

Step 4: Return 0

```
return 0;
```

This means:

The stack is not full.

Return Values

Return value	Meaning
1	Stack is full. Heap allocation failed.
0	Stack is not full. Heap allocation succeeded.

Important Memory Note

If this happens:

```
struct Node *t = malloc(sizeof(struct Node));
```

and allocation succeeds, then memory has been reserved.

If we do not call:

```
free(t);
```

we waste memory.

That is called a memory leak.

Complexity

`isFull()` is usually treated as:

```
O(1)
```

Reason:

It tries to allocate one node only.

It does not scan the stack.

10. `push(x)` Operation

Meaning

`push(x)` inserts a new value at the top of the stack.

Example before push:

```
top -> 20 -> 10 -> NULL
```

Call:

```
push(30);
```

After push:

```
top -> 30 -> 20 -> 10 -> NULL
```

The new value `30` becomes the top.

11. `push(x)` Code

```
void push(int x) {
    struct Node *t;

    t = (struct Node *)malloc(sizeof(struct Node));

    if (t == NULL) {
        printf("Stack Overflow\n");
    } else {
        t->data = x;
        t->next = top;
        top = t;
    }
}
```

12. `push(x)` Code Explanation

Step 1: Create a temporary pointer

```
struct Node *t;
```

`t` will point to the new node.

Step 2: Allocate memory for the new node

```
t = (struct Node *)malloc(sizeof(struct Node));
```

This asks the heap for enough memory to store one `struct Node`.

Step 3: Check if memory allocation failed

```
if (t == NULL) {  
    printf("Stack Overflow\n");  
}
```

If `malloc()` returns `NULL`, no node was created.

This means stack overflow.

Step 4: Store the value

```
t->data = x;
```

This stores the new value inside the new node.

Step 5: Link the new node to the old top

```
t->next = top;
```

This makes the new node point to the previous top node.

Step 6: Move `top` to the new node

```
top = t;
```

Now the new node becomes the top of the stack.

13. The Key Push Pattern

The most important push pattern is:

```
t->next = top;  
top = t;
```

Simple meaning:

1. The new node points to the old top.
2. top moves to the new node.

This order matters.

Correct Order

```
t->next = top;  
top = t;
```

Wrong Order

```
top = t;  
t->next = top;
```

The wrong order can damage the stack connection.

It may make the new node point to itself or lose the old list.

14. push(20) Dry Run

Start:

```
top -> NULL
```

Call:

```
push(20);
```

Step 1: Create new node

```
t -> [ ? | ? ]
```

Step 2: Store data

```
t->data = 20;
```

Now:

```
t -> [ 20 | ? ]
```

Step 3: Link new node to old top

```
t->next = top;
```

Since `top` is currently `NULL`:

```
t->next = NULL
```

Now:

```
t -> 20 -> NULL
```

Step 4: Move top

```
top = t;
```

Final stack:

```
top -> 20 -> NULL
```

15. More Push Dry Run

Start:

```
top -> NULL
```

Perform:

```
push(3);  
push(8);  
push(15);
```

After push(3)

```
top -> 3 -> NULL
```

After push(8)

The new node 8 points to the old top 3.

```
top -> 8 -> 3 -> NULL
```

After push(15)

The new node 15 points to the old top 8.

```
top -> 15 -> 8 -> 3 -> NULL
```

Logical stack view:

```
Top -> 15  
      8  
      3
```

The newest value is 15, so 15 is the top.

16. pop() Operation

Meaning

pop() removes and returns the top value.

Example before pop:


```
top -> 30 -> 20 -> 10 -> NULL
```

Call:

```
pop();
```

The top value is:

```
30
```

After pop:

```
top -> 20 -> 10 -> NULL
```

Returned value:

```
30
```

17. `pop()` Code

```
int pop() {
    struct Node *p;
    int x = -1;

    if (top == NULL) {
        printf("Stack Underflow\n");
    } else {
        p = top;
        x = p->data;
        top = top->next;
        free(p);
    }

    return x;
}
```

18. pop() Code Explanation

Step 1: Create a temporary pointer

```
struct Node *p;
```

`p` will point to the node that will be deleted.

Step 2: Set default return value

```
int x = -1;
```

If the stack is empty, there is no real value to return.

So `-1` is used as a simple error value.

Important note:

If `-1` is also a valid stack value, this method can be confusing.

For beginner code, this simple method is acceptable.

Step 3: Check underflow

```
if (top == NULL) {  
    printf("Stack Underflow\n");  
}
```

If `top == NULL`, the stack is empty.

There is no node to remove.

Step 4: Save the old top node

```
p = top;
```

Now `p` points to the node that will be removed.

Step 5: Save the data

```
x = p->data;
```

This saves the top value so we can return it after deleting the node.

Step 6: Move `top` to the next node

```
top = top->next;
```

Now the second node becomes the new top.

Step 7: Free the old top node

```
free(p);
```

This releases the memory used by the removed node.

Step 8: Return the saved value

```
return x;
```

19. The Key Pop Pattern

The most important pop pattern is:

```
p = top;  
x = p->data;  
top = top->next;  
free(p);  
return x;
```

Simple meaning:

1. Save the old top node.
2. Save the value inside it.
3. Move top to the next node.

4. Free the old top node.
5. Return the saved value.

This order matters.

Do not free the node before moving `top` safely.

Wrong idea:

```
free(top);  
top = top->next;
```

This is unsafe because after freeing `top`, using `top->next` is invalid.

Correct idea:

```
p = top;  
top = top->next;  
free(p);
```

20. `pop()` Dry Run

Start:

```
top -> 15 -> 8 -> 3 -> NULL
```

Call:

```
pop();
```

Step 1: Check empty condition

```
top == NULL
```

This is false because `top` points to `15`.

Step 2: Save old top

```
p = top;
```

Now:

```
p points to node 15
```

Step 3: Save data

```
x = p->data;
```

So:

```
x = 15
```

Step 4: Move top

```
top = top->next;
```

Now `top` points to `8`:

```
top -> 8 -> 3 -> NULL
```

Step 5: Free old node

```
free(p);
```

The node containing `15` is deleted from heap memory.

Step 6: Return value

```
pop() returns 15
```

Final stack:

```
top -> 8 -> 3 -> NULL
```

21. `stackTop()` Operation

Meaning

`stackTop()` reads the current top value without deleting it.

It is similar to:

```
peek(1)
```

But it is simpler because it only reads the first node.

22. `stackTop()` Code

```
int stackTop() {  
    if (top == NULL) {  
        return -1;  
    }  
  
    return top->data;  
}
```

23. `stackTop()` Example

Given:

```
top -> 40 -> 30 -> 20 -> NULL
```

Call:

```
stackTop();
```

Since `top` points to `40`, the function returns:

40

The stack does not change.

Before:

`top -> 40 -> 30 -> 20 -> NULL`

After:

`top -> 40 -> 30 -> 20 -> NULL`

Difference Between `pop()` and `stackTop()`

Operation	Returns top value?	Removes top value?	Changes stack?
<code>pop()</code>	Yes	Yes	Yes
<code>stackTop()</code>	Yes	No	No

24. `peek(pos)` Operation

Meaning

`peek(pos)` reads a value at a position from the top without deleting it.

The position is counted from the top.

Example:

`top -> 15 -> 8 -> 3 -> NULL`
 1 2 3

So:

```
peek(1) returns 15  
peek(2) returns 8  
peek(3) returns 3
```

Important:

```
peek() only reads.  
It does not remove anything.
```

25. Why Linked-List `peek(pos)` Needs Traversal

This is one of the most important ideas in Lesson 6.

In an array stack, we can directly jump to an index.

For array stack:

```
index = top - pos + 1;
```

Then we can directly access:

```
S[index]
```

That is possible because arrays support direct indexing.

But linked lists do not support direct indexing.

A linked list is connected node by node.

Example:

```
top -> 15 -> 8 -> 3 -> NULL
```

If we want position 3, we cannot jump directly to `3`.

We must move like this:


```
15 -> 8 -> 3
```

So linked-list `peek(pos)` must traverse.

That means:

```
It moves from one node to the next node until it reaches the requested position.
```

26. Why Move `pos - 1` Times?

In `peek(pos)`, the pointer starts at the top.

The top is already position 1.

Example:

```
top -> 15 -> 8 -> 3 -> NULL
      1   2   3
```

If we want `peek(1)`, we move:

```
0 times
```

Because we are already at position 1.

If we want `peek(2)`, we move:

```
1 time
```

From `15` to `8`.

If we want `peek(3)`, we move:

```
2 times
```

From `15` to `8`, then from `8` to `3`.

So the rule is:

Number of moves = pos - 1

27. peek(pos) Code

```
int peek(int pos) {
    struct Node *p = top;
    int i;

    if (pos <= 0) {
        return -1;
    }

    for (i = 0; p != NULL && i < pos - 1; i++) {
        p = p->next;
    }

    if (p != NULL) {
        return p->data;
    }

    return -1;
}
```

28. peek(pos) Code Explanation

Step 1: Start from the top

```
struct Node *p = top;
```

`p` is a temporary pointer.

It starts at the top node.

Step 2: Check invalid position

```
if (pos <= 0) {  
    return -1;  
}
```

Positions must start from 1.

So these are invalid:

```
peek(0)  
peek(-1)  
peek(-5)
```

Step 3: Move `pos - 1` times

```
for (i = 0; p != NULL && i < pos - 1; i++) {  
    p = p->next;  
}
```

This loop moves node by node.

It stops if:

- we have moved enough times, or
- `p` becomes `NULL`

If `p` becomes `NULL`, the requested position does not exist.

Step 4: Return data if valid

```
if (p != NULL) {  
    return p->data;  
}
```

If `p` points to a node, return that node's data.

Step 5: Return `-1` if invalid

```
return -1;
```

This means the position was invalid or too large.

29. `peek(3)` Dry Run

Start:

```
top -> 15 -> 8 -> 3 -> NULL
      1    2    3
```

Call:

```
peek(3);
```

Step 1: Start pointer at top

```
p points to 15
```

Step 2: Calculate number of moves

```
pos - 1 = 3 - 1 = 2
```

So we move 2 times.

Move 1

```
p moves from 15 to 8
```

Move 2

```
p moves from 8 to 3
```

Now:

```
p points to 3
```

So:

```
return p->data;
```

returns:

```
3
```

Final result:

```
peek(3) returns 3
```

The stack does not change.

30. Invalid `peek(pos)` Examples

Given:

```
top -> 15 -> 8 -> 3 -> NULL
      1    2    3
```

Example 1: `peek(0)`

Position `0` is invalid because counting starts from 1.

Result:

```
-1
```

Example 2: `peek(4)`

Position 4 does not exist.

Trace:

```
p starts at 15  
move 1: p goes to 8  
move 2: p goes to 3  
move 3: p goes to NULL
```

Now `p == NULL`.

Result:

-1

31. Complete Lesson 6 C Program

```
#include <stdio.h>  
#include <stdlib.h>  
  
struct Node {  
    int data;  
    struct Node *next;  
};  
  
struct Node *top = NULL;  
  
int isEmpty() {  
    return top == NULL;  
}  
  
int isFull() {  
    struct Node *t = (struct Node *)malloc(sizeof(struct Node));  
  
    if (t == NULL) {  
        return 1;  
    }  
  
    free(t);  
    return 0;  
}  
  
void push(int x) {  
    struct Node *t;
```

```

t = (struct Node *)malloc(sizeof(struct Node));

if (t == NULL) {
    printf("Stack Overflow\n");
} else {
    t->data = x;
    t->next = top;
    top = t;
}
}

int pop() {
    struct Node *p;
    int x = -1;

    if (top == NULL) {
        printf("Stack Underflow\n");
    } else {
        p = top;
        x = p->data;
        top = top->next;
        free(p);
    }

    return x;
}

int peek(int pos) {
    struct Node *p = top;
    int i;

    if (pos <= 0) {
        return -1;
    }

    for (i = 0; p != NULL && i < pos - 1; i++) {
        p = p->next;
    }

    if (p != NULL) {
        return p->data;
    }

    return -1;
}

int stackTop() {

```

```

    if (top == NULL) {
        return -1;
    }

    return top->data;
}

int main() {
    push(3);
    push(8);
    push(15);

    printf("Top element: %d\n", stackTop());
    printf("Peek position 3: %d\n", peek(3));
    printf("Popped value: %d\n", pop());
    printf("Top after pop: %d\n", stackTop());

    return 0;
}

```

32. Full Program Dry Run

Program operations:

```

push(3);
push(8);
push(15);
stackTop();
peek(3);
pop();
stackTop();

```

Initial State

top -> NULL

Operation 1: push(3)

Create node 3.

Link it to old top, which is NULL.

Move `top` to node `3`.

Result:

```
top -> 3 -> NULL
```

Operation 2: `push(8)`

Create node `8`.

Link it to old top, which is node `3`.

Move `top` to node `8`.

Result:

```
top -> 8 -> 3 -> NULL
```

Operation 3: `push(15)`

Create node `15`.

Link it to old top, which is node `8`.

Move `top` to node `15`.

Result:

```
top -> 15 -> 8 -> 3 -> NULL
```

Operation 4: `stackTop()`

`top` points to `15`.

So:

```
stackTop() returns 15
```

The stack does not change.

Operation 5: peek(3)

Stack:

```
top -> 15 -> 8 -> 3 -> NULL
      1    2    3
```

Position 3 is 3.

So:

peek(3) returns 3

The stack does not change.

Operation 6: pop()

Top value is 15.

Remove node 15.

Move top to 8.

Free node 15.

Result:

```
top -> 8 -> 3 -> NULL
```

Return value:

15

Operation 7: stackTop()

Now top points to 8.

So:

```
stackTop() returns 8
```

33. Expected Output

The program prints:

```
Top element: 15
Peek position 3: 3
Popped value: 15
Top after pop: 8
```

34. Dry Run Summary Table

Step	Operation	Return/output	Stack after operation
0	Start	—	top -> NULL
1	push(3)	—	top -> 3 -> NULL
2	push(8)	—	top -> 8 -> 3 -> NULL
3	push(15)	—	top -> 15 -> 8 -> 3 -> NULL
4	stackTop()	returns 15	top -> 15 -> 8 -> 3 -> NULL
5	peek(3)	returns 3	top -> 15 -> 8 -> 3 -> NULL
6	pop()	returns 15	top -> 8 -> 3 -> NULL
7	stackTop()	returns 8	top -> 8 -> 3 -> NULL

35. Complexity of Lesson 6 Operations

Operation	Time complexity	Auxiliary space	Reason
push()	$O(1)$	$O(1)$	Adds one node at the front.
pop()	$O(1)$	$O(1)$	Removes one node from the front.
stackTop()	$O(1)$	$O(1)$	Reads <code>top->data</code> directly.

Operation	Time complexity	Auxiliary space	Reason
<code>isEmpty()</code>	$O(1)$	$O(1)$	Checks whether <code>top == NULL</code> .
<code>isFull()</code>	$O(1)$	$O(1)$	Tries to allocate one node.
<code>peek(pos)</code>	$O(n)$	$O(1)$	May traverse node by node.

36. Why `push()` Is $O(1)$

`push()` does a fixed number of steps:

1. Allocate one node.
2. Store data.
3. Link new node to old top.
4. Move top.

It does not scan the full list.

So:

`push() =  $O(1)$`

37. Why `pop()` Is $O(1)$

`pop()` directly removes the node pointed to by `top`.

It does not search for the top.

It already knows where the top is:

`top`

So:

`pop() =  $O(1)$`

38. Why peek(pos) Is O(n)

peek(pos) may need to move through many nodes.

Example:

```
top -> 50 -> 40 -> 30 -> 20 -> 10 -> NULL
      1    2    3    4    5
```

If we call:

```
peek(5);
```

The pointer must move:

```
50 -> 40 -> 30 -> 20 -> 10
```

That takes multiple steps.

If there are n nodes, the worst case may require moving through n nodes.

So:

```
peek(pos) = O(n)
```

39. Array peek() vs Linked-List peek()

Feature	Array stack	Linked-list stack
Access style	Direct index access	Node-by-node traversal
Formula	$\text{index} = \text{top} - \text{pos} + 1$	Move $\text{pos} - 1$ times
Time complexity	$O(1)$	$O(n)$

Important comparison:

Array peek is faster because arrays allow direct indexing.
Linked-list peek is slower because linked lists require traversal.

40. Heap Memory in Lesson 6

A linked-list stack uses heap memory.

Every successful `push()` creates a new node using:

```
malloc()
```

Every successful `pop()` deletes a node using:

```
free()
```

So memory grows and shrinks during program execution.

During Push

```
t = (struct Node *)malloc(sizeof(struct Node));
```

This creates memory for one new node.

During Pop

```
free(p);
```

This releases memory for the removed node.

During isFull

```
struct Node *t = malloc(sizeof(struct Node));
```

This tests whether memory is available.

If memory is available, we must release it:

```
free(t);
```

41. Overflow and Underflow

Overflow

Overflow means trying to push when no memory is available.

In a linked-list stack, overflow happens when:

```
malloc() returns NULL
```

Example:

```
t = (struct Node *)malloc(sizeof(struct Node));

if (t == NULL) {
    printf("Stack Overflow\n");
}
```

Underflow

Underflow means trying to pop when the stack is empty.

In a linked-list stack, underflow happens when:

```
top == NULL
```

Example:

```
if (top == NULL) {
    printf("Stack Underflow\n");
}
```

42. Array Overflow vs Linked-List Overflow

Stack type	Overflow condition	Simple meaning
Array stack	<code>top == size - 1</code>	Array has no empty slot.
Linked-list stack	<code>malloc()</code> returns <code>NULL</code>	Heap cannot create a new node.

43. Array Underflow vs Linked-List Underflow

Stack type	Underflow condition	Simple meaning
Array stack	<code>top == -1</code>	No active array element.
Linked-list stack	<code>top == NULL</code>	No node exists.

44. Operation Effects Summary

Operation	What it does	Changes stack?
<code>push(x)</code>	Inserts <code>x</code> at the top.	Yes
<code>pop()</code>	Removes and returns the top value.	Yes
<code>peek(pos)</code>	Reads a value at a position from the top.	No
<code>stackTop()</code>	Reads the current top value.	No
<code>isEmpty()</code>	Checks if stack has no nodes.	No
<code>isFull()</code>	Checks if heap can create a new node.	No real stack change

Note:

`isFull()` temporarily allocates a test node, but it frees it immediately.

So the actual stack does not change.

45. Common Beginner Mistakes

Mistake 1: Thinking `top` is still an index

Wrong idea:

```
top stores an index
```

Correct idea:

```
In a linked-list stack, top is a pointer.
```

In an array stack:

```
top == -1
```

In a linked-list stack:

```
top == NULL
```

Mistake 2: Using `top == -1` in a linked-list stack

Wrong:

```
if (top == -1)
```

Correct:

```
if (top == NULL)
```

Reason:

```
top
```

 is a pointer, not an integer.

Mistake 3: Reversing the push links

Wrong:

```
top = t;  
t->next = top;
```

Correct:

```
t->next = top;  
top = t;
```

Reason:

The new node must first point to the old top.

Then `top` can move to the new node.

Mistake 4: Forgetting to check `malloc()`

Wrong:

```
t = (struct Node *)malloc(sizeof(struct Node));  
t->data = x;
```

Problem:

If `malloc()` fails, `t` is `NULL`.

Then `t->data` is unsafe.

Correct:

```
t = (struct Node *)malloc(sizeof(struct Node));  
  
if (t == NULL) {  
    printf("Stack Overflow\n");  
} else {  
    t->data = x;  
    t->next = top;
```

```
    top = t;
}
```

Mistake 5: Forgetting underflow check before `pop()`

Wrong:

```
p = top;
x = p->data;
```

Problem:

If `top == NULL`, then `p` is also `NULL`.

Accessing `p->data` is invalid.

Correct:

```
if (top == NULL) {
    printf("Stack Underflow\n");
}
```

Mistake 6: Freeing the node too early

Wrong:

```
free(top);
top = top->next;
```

Problem:

After freeing `top`, using `top->next` is unsafe.

Correct:

```
p = top;  
top = top->next;  
free(p);
```

Mistake 7: Forgetting to free popped nodes

Wrong idea:

```
Just move top and ignore the old node.
```

Problem:

The old node remains allocated in heap memory.

That causes a memory leak.

Correct:

```
free(p);
```

Mistake 8: Thinking linked-list `peek()` uses the array formula

Wrong for linked lists:

```
index = top - pos + 1;
```

Correct for linked lists:

```
Move a pointer pos - 1 times.
```

Reason:

Linked lists do not support direct indexing.

Mistake 9: Forgetting invalid positions in `peek(pos)`

Wrong idea:

`peek(0)` is allowed

Correct idea:

Positions start from 1.

So these are invalid:

`peek(0)`
`peek(-1)`
`peek(100)` when the stack has fewer nodes

Mistake 10: Forgetting to free the test node in `isFull()`

Wrong:

```
int isFull() {
    struct Node *t = malloc(sizeof(struct Node));

    if (t == NULL) {
        return 1;
    }

    return 0;
}
```

Problem:

If `malloc()` succeeds, the test node is never freed.

Correct:

```
int isFull() {
    struct Node *t = malloc(sizeof(struct Node));
```

```
    if (t == NULL) {  
        return 1;  
    }  
  
    free(t);  
    return 0;  
}
```

46. Important Rules to Memorize

Empty Condition

```
top == NULL
```

Push Pattern

```
t = (struct Node *)malloc(sizeof(struct Node));  
t->data = x;  
t->next = top;  
top = t;
```

Pop Pattern

```
p = top;  
x = p->data;  
top = top->next;  
free(p);  
return x;
```

Peek Traversal Rule

```
Move pos - 1 times from top.
```

Stack Top Rule

```
return top->data;
```

Linked-List Full Condition

```
malloc() returns NULL
```

Linked-List Underflow Condition

```
top == NULL
```

47. Short Glossary

Term	Simple meaning
Stack	A data structure that follows LIFO.
LIFO	Last In, First Out.
Linked-list stack	A stack implemented using linked-list nodes.
Node	One element in a linked list.
<code>data</code>	The value stored inside a node.
<code>next</code>	Pointer to the next node.
<code>top</code>	Pointer to the top node.
<code>NULL</code>	Means the pointer points to nothing.
<code>push(x)</code>	Insert <code>x</code> at the top.
<code>pop()</code>	Remove and return the top value.
<code>peek(pos)</code>	Read a value at a position from the top.
<code>stackTop()</code>	Read the current top value only.
<code>isEmpty()</code>	Check whether <code>top == NULL</code> .
<code>isFull()</code>	Check whether heap memory is available.
<code>malloc()</code>	Allocates heap memory.
<code>free()</code>	Releases heap memory.
Overflow	Trying to push when memory is unavailable.
Underflow	Trying to pop when the stack is empty.

Term	Simple meaning
Traversal	Moving node by node through a linked list.
Memory leak	Allocated memory that is not released.
<code>O(1)</code>	Constant time.
<code>O(n)</code>	Time grows with the number of nodes.

48. Checkpoint Questions

Try to answer these without looking at the answers.

1. What does `top == NULL` mean?
2. What are the two fields inside a linked-list node?
3. What does `data` store?
4. What does `next` store?
5. What does `push(x)` do?
6. What does `pop()` do?
7. What does `peek(pos)` do?
8. What does `stackTop()` do?
9. What does `isEmpty()` check?
10. What does `isFull()` check?
11. Why does linked-list `peek(pos)` need traversal?
12. How many times does `peek(pos)` move the pointer?
13. Why is linked-list `peek(pos)` `O(n)`?
14. Why is linked-list `push()` `O(1)`?
15. Why is linked-list `pop()` `O(1)`?
16. Why must `push()` check whether `malloc()` returned `NULL`?
17. Why must `pop()` check whether `top == NULL`?
18. Why must `pop()` save `p = top` before moving `top`?
19. Why must `pop()` call `free(p)`?
20. Why must `isFull()` call `free(t)` if allocation succeeds?

49. Checkpoint Answers

1. `top == NULL` means the stack is empty.
2. A node has `data` and `next`.
3. `data` stores the actual value.
4. `next` stores the address of the next node.
5. `push(x)` inserts `x` at the top.
6. `pop()` removes and returns the top value.

7. `peek(pos)` reads a value at a position from the top without deleting it.
 8. `stackTop()` reads the current top value only.
 9. `isEmpty()` checks whether `top == NULL`.
 10. `isFull()` checks whether memory is available for a new node.
 11. Linked-list `peek(pos)` needs traversal because linked lists do not allow direct indexing.
 12. `peek(pos)` moves the pointer `pos - 1` times.
 13. Linked-list `peek(pos)` is $O(n)$ because it may move through many nodes.
 14. `push()` is $O(1)$ because it adds one node at the front without scanning.
 15. `pop()` is $O(1)$ because it removes the top node directly.
 16. `push()` checks `malloc()` because memory allocation can fail.
 17. `pop()` checks `top == NULL` to avoid popping from an empty stack.
 18. `p = top` saves the old top node so we can free it safely later.
 19. `free(p)` releases the memory of the removed node.
 20. `isFull()` frees the test node to avoid a memory leak.
-

50. Practice Task 1: Trace Push Operations

Start:

```
top -> NULL
```

Perform:

```
push(4);  
push(9);  
push(2);
```

Step-by-Step Answer

After `push(4)`:

```
top -> 4 -> NULL
```

After `push(9)`:

```
top -> 9 -> 4 -> NULL
```

After `push(2)`:

```
top -> 2 -> 9 -> 4 -> NULL
```

Final stack:

```
top -> 2 -> 9 -> 4 -> NULL
```

51. Practice Task 2: Trace One Pop

Start:

```
top -> 2 -> 9 -> 4 -> NULL
```

Call:

```
pop();
```

The top value is:

```
2
```

So `pop()` returns:

```
2
```

After pop:

```
top -> 9 -> 4 -> NULL
```

The node containing `2` is freed.

52. Practice Task 3: Trace Peek

Start:

```
top -> 2 -> 9 -> 4 -> NULL
      1   2   3
```

Find:

```
peek(2);
```

`p` starts at `2`.

Move:

```
pos - 1 = 2 - 1 = 1 time
```

So `p` moves from `2` to `9`.

Result:

```
peek(2) returns 9
```

The stack does not change.

53. Mini Practice Trace

Trace this sequence:

```
push(5);
push(12);
push(7);
peek(2);
pop();
push(20);
stackTop();
pop();
```

Step-by-Step Answer

Step	Operation	Return value	Stack after operation
0	Start	—	top -> NULL
1	push(5)	—	top -> 5 -> NULL
2	push(12)	—	top -> 12 -> 5 -> NULL
3	push(7)	—	top -> 7 -> 12 -> 5 -> NULL
4	peek(2)	12	top -> 7 -> 12 -> 5 -> NULL
5	pop()	7	top -> 12 -> 5 -> NULL
6	push(20)	—	top -> 20 -> 12 -> 5 -> NULL
7	stackTop()	20	top -> 20 -> 12 -> 5 -> NULL
8	pop()	20	top -> 12 -> 5 -> NULL

Final stack:

```
top -> 12 -> 5 -> NULL
```

54. Final Lesson Summary

Lesson 6 completes the main operations for a linked-list stack.

The most important ideas are:

```
A linked-list stack uses nodes.
```

```
top is a pointer, not an index.
```

```
The empty condition is top == NULL.
```

```
push() adds a new node at the front.
```

`pop()` removes the front node and frees it.

`peek(pos)` must traverse because linked lists do not support direct indexing.

`stackTop()` reads `top->data` without deleting it.

`isFull()` checks whether heap memory can create a new node.

`push`, `pop`, `stackTop`, `isEmpty`, and `isFull` are $O(1)$.

`peek(pos)` is $O(n)$.

If you understand Lesson 6 clearly, you are ready for Lesson 7, where the linked-list stack code is assembled into a fuller C implementation with display and main testing.